

Towards Repository-Level Java to C++ Translation with Large Language Models

Anonymous Author(s)*

Abstract

Code translation—converting code from one programming language to another while preserving functionality—is critical for legacy system migration, cross-platform development, and ecosystem modernization. Traditional rule-based or manual approaches are limited in scope and efficiency. While large language models (LLMs) have shown promise in code generation, most existing research focuses on translating isolated methods or small files, leaving project-level translation—where accumulated language paradigm differences become significant—largely unexplored.

This paper proposes **TOOL**, a hierarchical, macro-to-micro translation framework designed to mitigate error propagation in large-scale code translation. The framework first performs static analysis to extract call graphs and decompose code into functional modules using a custom Java parser tool. Translation then proceeds in two layers: (1) at the class level, where overall architecture and header files are designed with language-specific pattern mapping, and (2) at the method level, where implementations are translated bottom-up along call chains to ensure dependency stability. To bridge library and runtime differences, the framework dynamically generates necessary dependencies and an autonomous agent iteratively compiles, tests, and repairs the translated code using intelligent error analysis and fix strategies.

We evaluate **TOOL** by translating several cohesive Java modules to C++, two languages with significant differences in underlying execution models. Experimental results demonstrate that ... **TODO:** **Add experimental results**

Keywords: Code translation, Large language models, Project-level translation, Java, C++, Static analysis, Hierarchical translation

ACM Reference Format:

Anonymous Author(s). 2026. Towards Repository-Level Java to C++ Translation with Large Language Models. In . ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

The increasing need for software migration, cross-language interoperability, and multi-platform deployment has made automated code translation a valuable research area. Early techniques relied on handcrafted rules or template-based transformations, which are difficult to scale. Recent advances in LLMs have enabled more flexible and context-aware code generation, yet their application

in translation remains largely confined to small, self-contained code snippets. Translating entire projects or interconnected modules introduces challenges such as architectural mismatch, dependency management, and compounding semantic gaps between languages—especially those with different runtime paradigms like Java (managed, JVM-based) and C++ (unmanaged, native).

Our work is situated at this intersection. We aim to develop a translation framework that leverages the generative strength of LLMs while deeply integrating extensible, domain-aware knowledge and validation mechanisms. We target translation tasks involving complex algorithm logic and API migrations (akin to Type 3/4 in the taxonomy by Jiao et al. [8]). Inspired by hybrid systems like IRENE [11] and compositional approaches like AlphaTrans [5], we propose a more generalized and extensible translation knowledge orchestration method. This approach is designed to dynamically retrieve and apply relevant constraints and patterns, ensuring high semantic fidelity and project-level usability without sacrificing flexibility.

This paper addresses the problem of error accumulation in project-level translation. We argue that translating files or methods independently without a high-level understanding of the module’s purpose leads to irreconcilable inconsistencies. Instead, we propose a top-down understanding followed by bottom-up translation, where the LLM first comprehends the macro-structure of a functional module, plans the overall translation strategy, and then executes translation in a layered manner.

Our main contributions are:

- **A hierarchical translation framework (**tool**)** that separates class-level design from method-level implementation, enabling macro-to-micro translation with reduced error propagation.
- **A call-graph-guided, bottom-up translation order** that stabilizes dependencies by translating leaf methods first, ensuring callees are available when translating callers.
- **An autonomous compilation and repair agent** that validates and corrects the translated code using intelligent error analysis, supporting iterative compilation with automated fix strategies.

2 Related Work

2.1 Large Language Model Hallucination

Rawte et al. [17] categorize hallucinations in large foundation models into four modalities: text, image, video, and audio, and propose evaluation criteria for assessing their severity. In this work, we focus specifically on text hallucinations generated by large language models. Hallucination constitutes a security-relevant risk in large language models, as Yao et al. [23] demonstrate that adversarial-like nonsensical inputs can provoke hallucinated outputs. To effectively detect hallucinations, Chen et al. [2] propose a robust discriminator named ReID to effectively detect hallucination in LLMs’ generated

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference’17, July 2017, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

answers based on a bilingual question-answering dialogue dataset RelQA. Manakul et al. [13] proposes SelfCheckGPT, which leveraging the idea that if an LLM has knowledge of a given concept, sampled responses are likely to be similar and contain consistent facts. To effectively mitigate hallucination, Ji et al. proposes to mitigate LLM hallucination via self-reflection [7], while Roychowdhury et al. [18] present the three major stages in designing hallucination-minimized LLM-based solutions in the financial domain.

Given that code is a safety and functionality-critical output of LLMs, researchers have increasingly focused on hallucinations in generated code. Liu et al. [9] conduct an empirical study of hallucinations in LLM-generated code and identify three primary types: requirement conflicts, knowledge hallucinations, and code inconsistencies. Their results reveal severe consequences, where a dominant proportion of hallucinations lead to functional errors, with hallucinated code achieving a low pass@1 rate. Extending this line of work to repository-level generation, Zhang et al. [26] further categorize hallucinations into task requirement conflicts, factual knowledge conflicts, and project context conflicts. They also analyze the causes of these hallucinations, attributing them to factors such as low-quality training data, limitations in intent comprehension, and insufficient context awareness. To tackle the hallucination in generated code, Zhang et al. [26] demonstrates the effectiveness of retrieval-augmented generation, improving pass@1 scores across all models. **Other approaches include self-refinement prompts, chain-of-thought reasoning, and specialized code models. CITATION?**

2.2 Code Translation

Due to the rich syntactic structure of code, early seq-to-seq approaches frequently generated syntactically invalid outputs. To address this, Chen et al. [1] use of tree-to-tree neural networks, which encode and decode abstract syntax trees (ASTs) to explicitly model program structure. Drissi et al. [3] further refine this approach by introducing a grammar-driven tree-to-tree model that hard-codes target language syntax rules into the decoder, guaranteeing grammatical correctness. However, these works were limited in generalization and scalability by their dependence on parsers, and manually defined grammars.

As the field evolves, data-driven and unsupervised methods take center stage, reducing reliance on handcrafted rules and expert knowledge. Roziere et al. [19] introduced TransCoder, which leverages back-translation and denoising auto-encoding on monolingual code corpora to learn cross-lingual code representations without supervision. Meanwhile, pre-trained models such as CodeBERT and CodeT5 leverage large-scale code corpora to capture semantic and syntactic patterns via general-purpose representation learning, significantly improving code understanding and generation. However, Malyala et al. [12] found that those models also exhibit systematic errors in complex translation scenarios, such as type conversions and loop restructuring. Pan et al. [16] show that LLMs introduce a significant number of bugs during translation, highlighting the limitations of statistical approaches in understanding deep semantic and syntactic constraints.

Recently, research emphasizes augmenting LLMs with programming language-specific knowledge, such as type systems, safety

rules, and API mappings. Luo et al. [11] proposed the IRENE framework, which combines static analysis to extract “high-risk patterns” with retrieval-augmented generation to guide the model toward producing Rust code that complies with ownership rules. Similarly, Hong et al. [4] focused on type migration, employing a call-graph-based strategy to coordinate interface changes across functions. These hybrid systems effectively address the “semantic drift” and safety violations common in naive LLM translations. Extending this neuro-symbolic approach, Ibrahimzada et al. [5] introduced AlphaTrans, which compositionally combines symbolic knowledge with neural models for repository-level translation and validation. Meanwhile, Nitin et al. [14] enhances translation by generating multi-modal specifications (e.g., comments, tests) to provide additional context. While these methods significantly improve reliability, they often require substantial engineering effort to construct and maintain domain-specific knowledge bases.

2.3 Repository Translation

As capabilities of the LLMs grow, the research scope has expanded from function-level snippets to more complex, practical scenarios. Repository-level code translation presents unique challenges, including managing inter-file dependencies, build configurations, and project-wide context. Wang et al. [22] introduced RepoTransBench, a large-scale, multi-language benchmark for evaluating repository-level translation, revealing that even state-of-the-art LLMs struggle with the overall success rates. To tackle this, Zhang et al. [25] proposed a skeleton-guided translation framework that first extracts a high-level structural skeleton in the target language before filling in implementation details. Yuan et al. [24] and Ou et al. [15] further explored project-level translation by synergistically integrating knowledge graphs with the LLM in the repository context. In parallel, Liu et al. [10] attempt to improve the human-in-the-loop experience. They developed hmCodeTrans, an interactive translation framework that allows developers to provide real-time feedback, significantly reducing human effort compared to traditional post-editing workflows.

3 Empirical Study

We conduct an empirical study to systematically examine the effectiveness and limitations of large language models for repository-level Java-to-C++ translation.

- **RQ1:** How is the effectiveness of large language models in repository-level Java-to-C++ translation?
- **RQ2:** What are the common error patterns in large language model-driven repository-level Java-to-C++ translation?

3.1 Setup

Initial Project and Modules. We retrieved open-source Java projects from GitHub using the query expression “language:java size:<5000 sort:stars-desc”. This query specifies Java as the primary programming language, restricts the repository size to under 5,000 KB to facilitate manual analysis, and ranks the results in descending order by star count to prioritize more popular projects. We collected projects from the first XX pages of the search results and excluded those unsuitable for C++ translation: (1) educational or tutorial projects containing toy examples; (2) Android applications

with platform-specific dependencies; and (3) frameworks with sophisticated design patterns that is hard to create counterparts in C++ (e.g., the Spring framework which is annotation-intensive that heavily rely on reflection). As a result, we collected six open-source projects spanning diverse domains:

- **failsafe**¹: A fault tolerance library for building resilient distributed systems.
- **jvm-sandbox**²: An AOP (Aspect-Oriented Programming) framework for runtime code manipulation.
- **easyexcel**³: A library for Excel file processing with simplified APIs.
- **httpclient**⁴: An HTTP client library for network communication.
- **hutool**⁵: A comprehensive Java utility library.
- **Java-WebSocket**⁶: A WebSocket implementation for real-time communication.

As the project may have multiple independent and separated modules, we perform module analysis to aggregate files into modules. Specifically, we iterate each Java source file, parse each Java source file into the AST representation using JavaParser [6], and obtain their import statements as well as the used types that do not need import statements in the same directory. We resolve the imported types into the corresponding Java source file. Upon completion of this process, the interconnected source files are partitioned into multiple clusters, each representing a logically coherent module that implements a specific and self-contained functionality. Overall, we identified 24 modules encompassing diverse source files and methods. A detailed summary of the dataset is presented in Table 2.

Selected Modules. For our empirical study, we randomly sampled three representative modules. i.e., EnableJUnit4MigrationSupport from junit5, Cookie from httpclient, and CircuitBreakerExecutor from failsafe. The selected modules represent varying levels of scale, including a small-scale module (8 files and 93 methods in Cookie), a medium-scale module (23 files and 141 methods in CircuitBreakerExecutor), and a large-scale module (32 files and 234 methods in EnableJUnit4MigrationSupport). In total, the three modules include 468 methods. Based on a standard sample size calculation [21], this sample size ensures a 95% confidence level with a 4.2% margin of error relative to the overall population of 3,165 methods.

Subject LLMs. We selected three popular and state-of-the-art large language models. i.e., **DeepSeek-V3.2**, **Qwen-3.5Plus** and **ChatGPT-5.1**.

Translation Strategies. As an entire module may exceed the model’s context window limitations, segmentation is required prior to translation. We evaluate two representative segmentation strategies that reflect common developer practices:

- **File-by-file translation:** Each Java source file is translated independently as a self-contained unit. This strategy preserves the

¹<https://github.com/failsafe-lib/failsafe>

²<https://github.com/alibaba/jvm-sandbox>

³<https://github.com/alibaba/easyexcel>

⁴<https://github.com/wso2/wso2-commons-httpclient>

⁵<https://github.com/dromara/hutool>

⁶<https://github.com/TooTallNate/Java-WebSocket>

Project	Root Class of Module	File (#)	Method (#)
failsafe	BulkheadBuilder.java	25	140
	CircuitBreakerExecutor.java	23	141
	DelayablePolicy.java	24	123
	ExecutionImpl.java	22	149
	FailsafeExecutor.java	24	170
	FailurePolicy.java	25	132
	RateLimiterBuilder.java	25	155
	RateLimiterExecutor.java	24	146
	RetryPolicyConfig.java	23	133
	SyncExecutionImpl.java	24	142
jvm-sandbox	TimeoutExecutor.java	21	116
	UndeclaredThrowableStrategy.java	11	62
	AdviceListenerTestCase.java	23	205
	ClassStructureByChildClassTestCase.java	12	89
	DebugLogExceptionModule.java	16	138
	EventWatchBuilderTestCase.java	18	174
	OrGroupFilter.java	13	138
easyexcel	ProgressPrinter.java	8	37
	TestIssues217.java	22	193
Java-WebSocket	TestIssues256.java	18	170
	ReadRowHolder.java	7	16
httpclient	ReadSheetHolder.java	11	51
	PerMessageDeflateExtensionTest.java	7	123
hutool	Draft_6455Test.java	19	109
	Cookie.java	8	93
junit	JSONParser.java	6	20
	EnableJUnit4MigrationSupport.java	32	246
Total		469	3,165

Table 1: Statistics of our Collected Projects and Modules (Module Root Class denotes the root class for running the module)

class structure, member declarations, and intra-file method relationships; however, it may overlook dependencies across files. XXX

- **Method-by-method translation:** Each Java method is translated in isolation, enabling finer-grained control over the translation process but sacrificing contextual information related to class structure and inter-method dependencies. XXX

RQ1 Setup. To address translation effectiveness (RQ1), we implemented a three-step evaluation pipeline. First, we translated all Java files in the selected modules using both strategies using file-by-file translation, and all Java methods using method-by-method translation, including member variables of each class that are translated separately. Then, we compiled the translated Java file using clang++ [20] 20.1.8 compiler with -c flag. For the translated methods, we concatenated the translated methods along with translated member variables into the C++ file and then used the clang++ compiler. Finally, we recorded the compilation outcomes (success/error) generated from clang++. A file is considered successfully compiled

Root Class of Module	Java File (#)	Java Method (#)	Translated CPP/Header File (#)
CircuitBreakerExecutor.java	28	141	13/23
Cookie	8	93	7/8
EnableJUnit4MigrationSupport.java	32	246	21/32
Total	63	480	41/63

Table 2: Translated Results for our Selected Models

when the compiler produces a corresponding object file (.o) without fatal errors.

RQ2 Setup. To analyze error patterns (RQ2), we conducted systematic manual analysis of compilation errors. For files that failed compilation, we retain individual method implementations and temporarily remove the rest methods and compiled it to isolate specific failure points. Five of the annotators perform open card sorting where they iteratively refine categories based on recurring patterns in compiler error messages. They keywords from clang++ output (e.g., “undefined reference”, “no matching function”, “type mismatch”) provided meaningful semantic cues. The same procedure is applied to methods translated under the method-by-method strategy. **The annotation process consisted of three phases. First, the annotators conducted pilot labeling on 200 cases to familiarize themselves with the task and establish initial categories. Second, they independently annotated the remaining cases, with each case subsequently reviewed and confirmed by another annotator. The basic units of error annotation were methods and classes: each method or class that failed to compile was annotated with one or more error types. However, for cases where the same problem in the same class caused multiple methods to fail compilation, we marked the error at the class level rather than marking it repeatedly at the method level. (Note the distinction between error level(class level vs. method level) and strategy type(file-by-file vs. method-by-file); we conducted statistical analysis for both error levels under each strategy.) The inter-annotator agreement measured by Cohen’s kappa was 0.78, indicating substantial agreement. In total, the annotation process required approximately 40 human-hours. Finally, we calculated the occurrence rate of each error type. Errors at class level and method level were analyzed separately to provide both macro-level (class structure) and micro-level (method implementation) insights.**

3.2 Translation Effectiveness (RQ1)

We measure the effectiveness of translation using the **compilation success rate (CSR)**, defined as the percentage of successfully compiled C++ files or methods relative to the total number of files or methods.

Model-wise Comparison. Figure ?? presents the overall effectiveness across different LLMs and segmentation strategies. Under the file-by-file strategy, where the entire source file is translated in a single step, ChatGPT-5.1 and DeepSeek-V3.2 achieve identical compilation success rates of 15.1%, while Qwen-3.5Plus performs slightly lower at 13.4%. It indicates that full-file translation remains

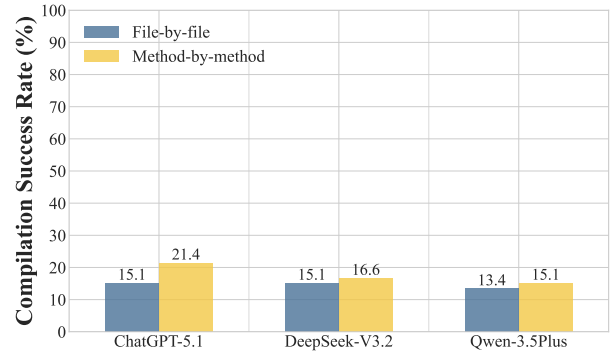


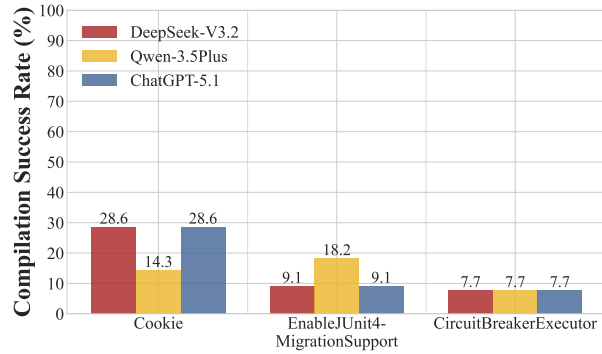
Figure 1: Effectiveness of Java-to-C++ Translation using LLMs under Two Translation Strategies

challenging for all models, with overall success rates below 16%. Under the method-by-method strategy, where individual functions are translated separately, performance improves for all models. ChatGPT-5.1 shows the most notable gain, increasing from 15.1% to 21.4%, representing the highest overall success rate in the figure. DeepSeek-V3.2 exhibits a modest improvement from 15.1% to 16.6%, while Qwen-3.5Plus rises from 13.4% to 15.1%.

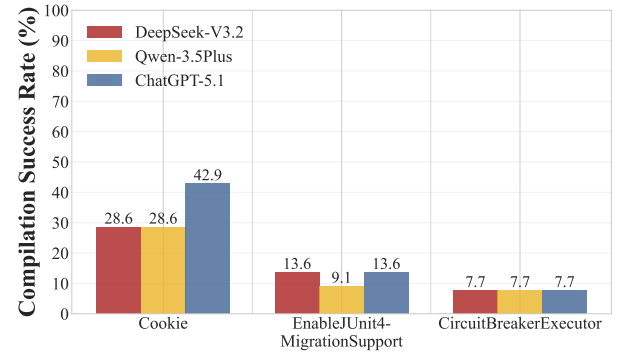
Overall, we can observe that method-by-method translation consistently outperforms file-by-file translation across all three models, suggesting that finer-grained decomposition reduces structural and dependency-related translation errors. Besides, ChatGPT-5.1 benefits the most from the granular strategy, showing a substantial improvement of 6.3 percentage points, whereas DeepSeek-V3.2 and Qwen-3.5Plus exhibit only moderate gains. However, despite these improvements, the overall compilation success rates remain relatively low, highlighting the persistent difficulty of fully automated Java-to-C++ translation using current LLMs.

Project-wise Comparison. Figure 2 further breaks down the compilation success rate (%) of C++-to-Java translation across three modules under two translation strategies. i.e., Cookie, EnableJUnit4MigrationSupport, and CircuitBreakerExecutor. Under the file-by-file translation strategy, the models exhibit different performance patterns across the three modules. For the Cookie module, DeepSeek-V3.2 and ChatGPT-5.1 achieve the highest compilation success rates at 28.6%, outperforming Qwen-3.5Plus, which reaches 14.3%. In contrast, for EnableJUnit4MigrationSupport, Qwen-3.5Plus performs best with 18.2%, while both DeepSeek-V3.2 and ChatGPT-5.1 obtain 9.1%. For CircuitBreakerExecutor, all three models show identical and relatively low performance, each achieving 7.7%. Under the method-by-method translation strategy, the performance distribution becomes more consistent across models. All three models achieve their highest success rates on the Cookie module, where ChatGPT-5.1 reaches a highest 42.9%, and DeepSeek and Qwen both achieve 28.6%. For EnableJUnit4MigrationSupport, the success rates decrease to 13.6% for DeepSeek-V3.2 and ChatGPT-5.1 and 9.1% for Qwen-3.5Plus. Similar to the file-by-file results, CircuitBreakerExecutor remains the most challenging module, with all models achieving the lowest performance at 7.7%.

Overall, these results highlight two main observations. First, different models demonstrate varying strengths across projects in



(a) file-by-file translation



(b) method-by-method translation

Figure 2: Effectiveness of Java-to-C++ Translation using LLMs with Translation Strategies w.r.t Project and Modules

Error Type	Error (#)	
	file-by-file translation	method-by-method translation
Language Syntax Violation	62	63
Type Error	81	110
Declaration Mismatch	2	2
Undefined Symbols	212	259
Duplicated Definitions	23	21
Header File Error	111	104
Total	494	560

Table 3: Taxonomy of Translation Errors

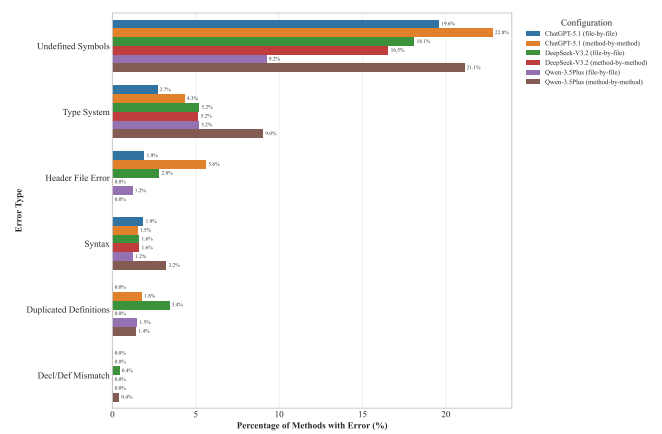


Figure 3: Error Type Distribution at Method Level Across Models and Strategies

both file-by-file and method-to-method translation, with lowest to 7.7% and highest to 42.9%. Second, method-by-method translation produces a noticeable performance gain for all models. The results suggests that the effectiveness of translation varies significantly across projects and models, and the project complexity plays an important role in translation difficulty. It indicates that specific code characteristics such as code structure or different usage of code syntax may influence translation success.

3.3 Error Pattern Analysis (RQ2)

To understand the common error patterns in LLM-driven code translation, we conducted a systematic analysis of compilation errors at both class level and method level under both translation strategies (file-by-file and method-by-method).

- **Language Syntax Violation:** Syntax and language feature errors.
- **Type Error:** Errors of type system, e.g., variable/parameter type mismatch; incorrect return value type; type undefined or incomplete; illegal type conversion, ...
- **Declaration Mismatch:** Mismatch between translated header file and source file.

- **Undefined Symbols:** Missing or undefined functions, variables, classes, etc.
- **Duplicated Definitions:** Redefinition of variables, functions, or classes.
- **Header File Error:** Errors of wrong building system, missing header files, wrong dependencies, etc.

Method-Level Error Analysis. Figure 3 presents the distribution of error types at the method level. The most prevalent error is MISSING/UNDEFINED SYMBOLS, affecting 17.90% of translated methods on average. This error occurs when the LLM generates code that references functions, classes, or standard library components that do not exist or are not properly included. The second most common error is TYPE SYSTEM ERROR (5.27%), which involves mismatches between expected and actual types, incorrect pointer conversions, or improper handling of C++ specific type features like const correctness and reference semantics. Other notable errors include BUILD/INCLUDE ERROR (1.93%), REDEFINITION ERROR (1.35%), and SYNTAX ERROR (0.82%).

Class-Level Error Analysis. Figure 4 reveals a markedly different error distribution at the class (file) level. The two most dominant errors are BUILD/INCLUDE ERROR (72.28%) and MISSING/UNDEFINED

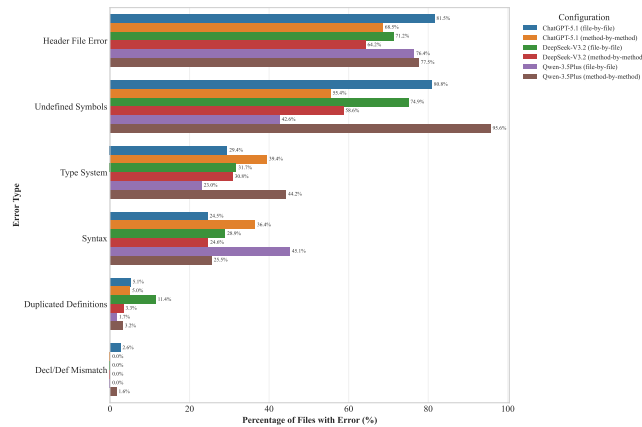


Figure 4: Error Type Distribution at Class Level Across Models and Strategies

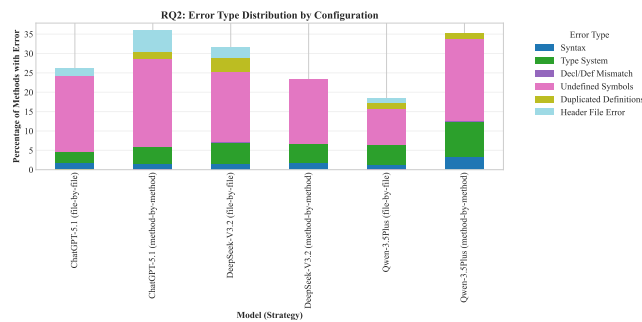


Figure 5: Stacked Error Distribution at Method Level

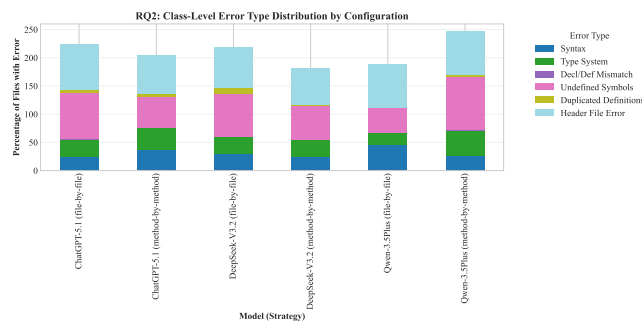


Figure 6: Stacked Error Distribution at Class Level

SYMBOLS (67.24%). The high prevalence of build/include errors at the class level indicates that LLMs frequently struggle with generating proper header guard structures, include directives, and headers across multiple files within a translation unit. TYPE SYSTEM ERROR remains significant at 32.50%, while TEMPLATE ERROR (12.11%) and SYNTAX ERROR (10.87%) also appear frequently. Notably, INHERITANCE/VIRTUAL ERROR (6.37%) and REDEFINITION ERROR (4.82%)

Model	Strategy	Missing Symbols (%)	Type System (%)	Build/Include (%)
DeepSeek-V3.2	File-by-file	18.1	5.2	2.8
	Method-by-method	16.5	5.2	0
Qwen-3.5Plus	File-by-file	9.2	5.2	1.2
	Method-by-method	21.1	9.0	0
ChatGPT-5.1	File-by-file	19.6	2.7	1.9
	Method-by-method	22.8	4.3	5.6

Table 4: Error Distribution by Model and Strategy (Method Level, Top 3 Error Types)

emerge as class-level concerns, reflecting challenges in translating complex C++ object-oriented features such as virtual function tables, multiple inheritance, and method overriding.

Comparison Across Models. Table 4 summarizes the error distribution by model and strategy at the method level. ChatGPT-5.1 exhibits the highest rate of missing symbol errors (22.8% under method-by-method), while DeepSeek-V3.2 shows relatively lower error rates across most categories. Qwen-3.5Plus demonstrates a distinctive pattern with higher type system error rates (9.0%) compared to other models.

Strategy Comparison. Regarding the translation strategy, method-by-method translation generally reduces class-level structural errors (e.g., build/include errors and template errors) since each method is translated independently without requiring file-level context. However, this strategy may increase method-level symbol resolution errors due to the lack of surrounding class context. File-by-file translation preserves class structure better but introduces more opportunities for compilation failures due to header organization and include directive issues.

Key Findings. The error pattern analysis reveals three key findings: (1) *Symbol resolution is the primary bottleneck:* The high prevalence of missing/undefined symbol errors across both levels indicates that LLMs struggle to maintain consistent symbol tables and API mappings during translation. (2) *C++ specific features remain challenging:* Template errors, inheritance/virtual function errors, and type system errors collectively account for a significant portion of failures, suggesting that LLMs need specialized handling for C++ language features that have no Java equivalents. (3) *File-level organization is a critical failure point:* The high rate of build/include errors at class level highlights the importance of proper header management and build system configuration—aspects that are often overlooked in translation but are essential for successful compilation.

4 Approach

This section presents the detailed design of TOOL, our hierarchical framework for project-level Java to C++ code translation. We first provide an overview of the framework architecture, then describe each component in detail.

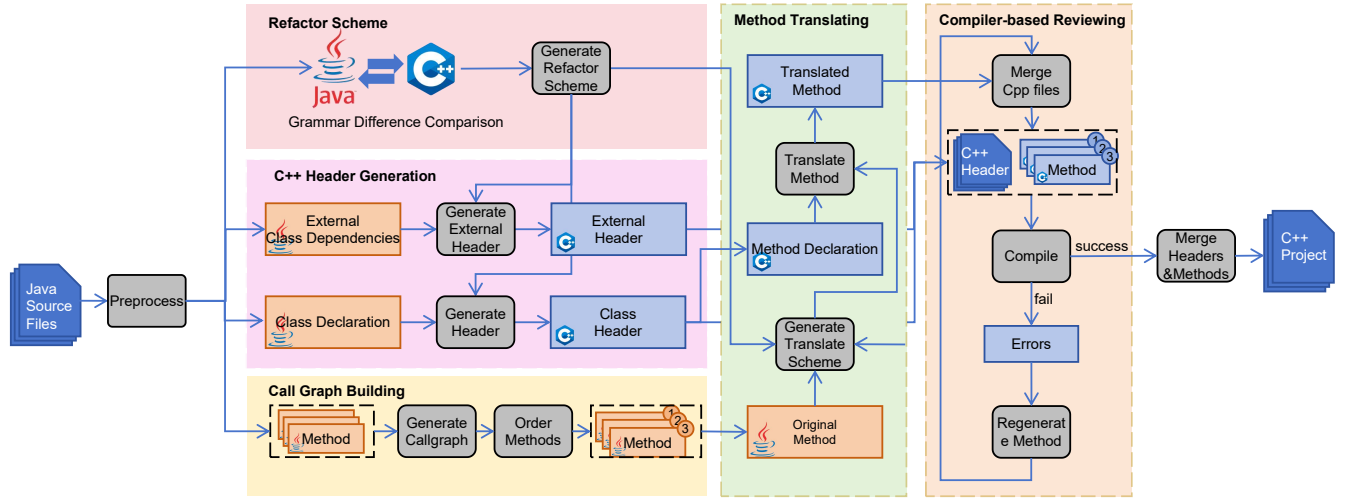


Figure 7: Overall architecture of TOOL showing the three-phase translation process.

4.1 Framework Overview

Figure 7 illustrates the overall architecture of TOOL. The framework adopts a **macro-to-micro translation strategy** that separates high-level structural design from low-level implementation details. This hierarchical decomposition addresses two key challenges in project-level code translation: (1) managing complexity at scale, and (2) mitigating error propagation through context-aware translation ordering.

The translation process consists of three main phases:

- (1) **Static Analysis Phase:** A custom Java parser tool extracts the project structure, including class hierarchies, method signatures, and call graph dependencies. This phase decomposes the codebase into a hierarchical graph representation comprising *Header nodes* (classes, interfaces, enums) and *Method nodes* (individual method implementations). The graph captures both structural relationships (inheritance, type dependencies) and behavioral relationships (method call dependencies), which guide subsequent translation ordering.
- (2) **Hierarchical Translation Phase:** The framework performs translation in two interdependent layers:
 - *Header-level translation:* For each Java class, interface, and enum, the LLM generates a translation scheme that maps Java language constructs to appropriate C++ patterns (e.g., interfaces to abstract base classes with virtual functions, checked exceptions to error code handling). This layer establishes the structural foundation and type mapping conventions for the entire project.
 - *Method-level translation:* Methods are translated following a bottom-up ordering derived from the call graph. Callee methods are translated before their callers,

enabling that already-translated code serves as reference for translating dependent methods. This dependency-aware approach significantly reduces translation inconsistencies and API mismatches.

- (3) **Validation and Repair Phase:** An autonomous compilation agent iteratively compiles the translated C++ code, analyzes compilation errors, and applies fixes using LLM-generated suggestions. The agent operates through a tool-use interface, invoking compilation and file modification primitives to iteratively resolve errors until compilation succeeds or maximum attempts are reached.

4.2 Static Analysis and Graph Construction

The first phase of TOOL involves parsing the Java source code and extracting structural information necessary for guiding the translation process. This phase constructs a project-level dependency graph that serves as the foundation for all subsequent translation decisions.

4.2.1 Project Graph Model. The static analysis phase constructs a bipartite graph representation consisting of two types of nodes:

- **Header Nodes:** Represent type declarations (classes, interfaces, enums). Each Header node captures:
 - The complete type declaration and field members
 - Inheritance relationships (parent class, implemented interfaces)
 - Import dependencies (external types required by this compilation unit)
 - A reference to all contained Method nodes
- **Method Nodes:** Represent individual method implementations. Each Method node captures:
 - The method signature (return type, parameter types, modifiers)
 - The complete method body implementation
 - Call dependencies (children: methods called by this method; parents: methods that call this method)

- External call references (calls to methods outside the project boundary)

This bipartite structure enables separate handling of structural translation (Headers) and behavioral translation (Methods), while maintaining explicit relationships between them.

4.2.2 Call Graph Extraction. To establish the correct translation order for methods, the framework extracts the complete call graph from the Java source code. We integrate `java-callgraph2`, a static analysis tool that identifies method call relationships through points-to analysis.

The call graph construction process involves:

- (1) Parsing method call traces from `java-callgraph2` output
- (2) Matching caller and callee signatures to Method nodes in the project graph
- (3) Establishing bidirectional edges between caller-callee pairs
- (4) Identifying external dependencies (calls to library methods outside the project)

The resulting call graph enables topological sorting of methods into translation layers, where all dependencies of a method in layer n are guaranteed to be in layers 1 through $n - 1$.

4.3 Hierarchical Translation Pipeline

The translation pipeline operates on the project graph in two distinct phases: Header-level translation establishes the structural foundation and type conventions, followed by Method-level translation that leverages dependency ordering for context propagation. Each phase employs a dual-pass strategy consisting of scheme generation followed by code generation.

4.3.1 Header-Level Translation. Header-level translation addresses the structural differences between Java and C++, establishing consistent conventions for the entire project before translating individual method implementations.

Scheme Generation. For each Header node, the LLM generates a **translation scheme**, a structured document that specifies:

- **Type mapping strategy:** How Java types map to C++ types (e.g., `ArrayList<T>` $\rightarrow$ `std::vector<T>`)
- **External dependency handling:** Identification of external dependencies that cannot be directly translated (e.g., third-party libraries, Java framework types), with strategies for mocking, replacing, or omitting them
- **Memory management strategy:** How to replace Java's garbage collection with appropriate C++ memory management (e.g., using stack allocation where possible, smart pointers for heap-allocated objects, RAII patterns for resource management)
- **Inheritance mapping:** How to map Java interfaces and abstract classes to C++ (e.g., interfaces $\rightarrow$ abstract base classes with pure virtual methods, default methods $\rightarrow$ virtual methods with implementations)
- **Generics conversion:** How to convert Java generics to C++ templates (including handling of bounded type parameters, wildcards, and type erasure implications)

The scheme serves as a contract between high-level design and detailed implementation, ensuring consistency across all methods within the class.

Header Translation. Using the generated scheme, the LLM translates each Header node to a C++ header file. The translation handles:

- Converting class declarations to C++ class syntax
- Translating field declarations with appropriate C++ types and initialization requirements
- Generating method declarations with translated signatures
- Adding necessary `#include` directives for both standard library and project dependencies
- Handling namespace declarations and forward declarations where appropriate

Importantly, in our framework, the translated C++ project structure may differ from the original Java structure while preserving core functionality. The dual-pass approach grants the LLM considerable flexibility during Header translation, particularly in scheme generation where structural refactoring decisions are made. For example:

- Java utility classes containing only static methods may be translated to C++ namespaces with free functions, eliminating the need for class instantiation
- Methods may be added, removed, or reorganized to better align with C++ idioms (e.g., separating declaration and definition, adding helper functions)
- Multiple Java classes in one file may be split into separate C++ header files, or vice versa, depending on compilation dependencies
- Interfaces may be merged or eliminated in favor of template-based designs where appropriate

This structural flexibility is particularly valuable when translating between languages with significant paradigmatic differences (e.g., object-oriented Java to multi-paradigm C++ with template metaprogramming), as it enables the translated code to follow idiomatic C++ patterns rather than mechanically mimicking Java's structure.

4.3.2 Method-Level Translation. Method-level translation focuses on translating implementation logic while maintaining behavioral equivalence. The key innovation is the bottom-up translation order, which enables effective context propagation.

Dependency-Aware Translation Ordering. Methods are grouped into layers using topological sort on the call graph:

- **Layer 1:** Leaf methods with no internal calls (pure computations, external library calls only)
- **Layer n :** Methods whose callees all reside in layers 1 through $n - 1$

Methods within the same layer have no mutual dependencies and can be translated in parallel, providing opportunities for concurrent processing.

Scheme Generation. For each Method node, the LLM generates a detailed translation scheme that includes:

- **Functionality summary:** A concise description of what the method accomplishes

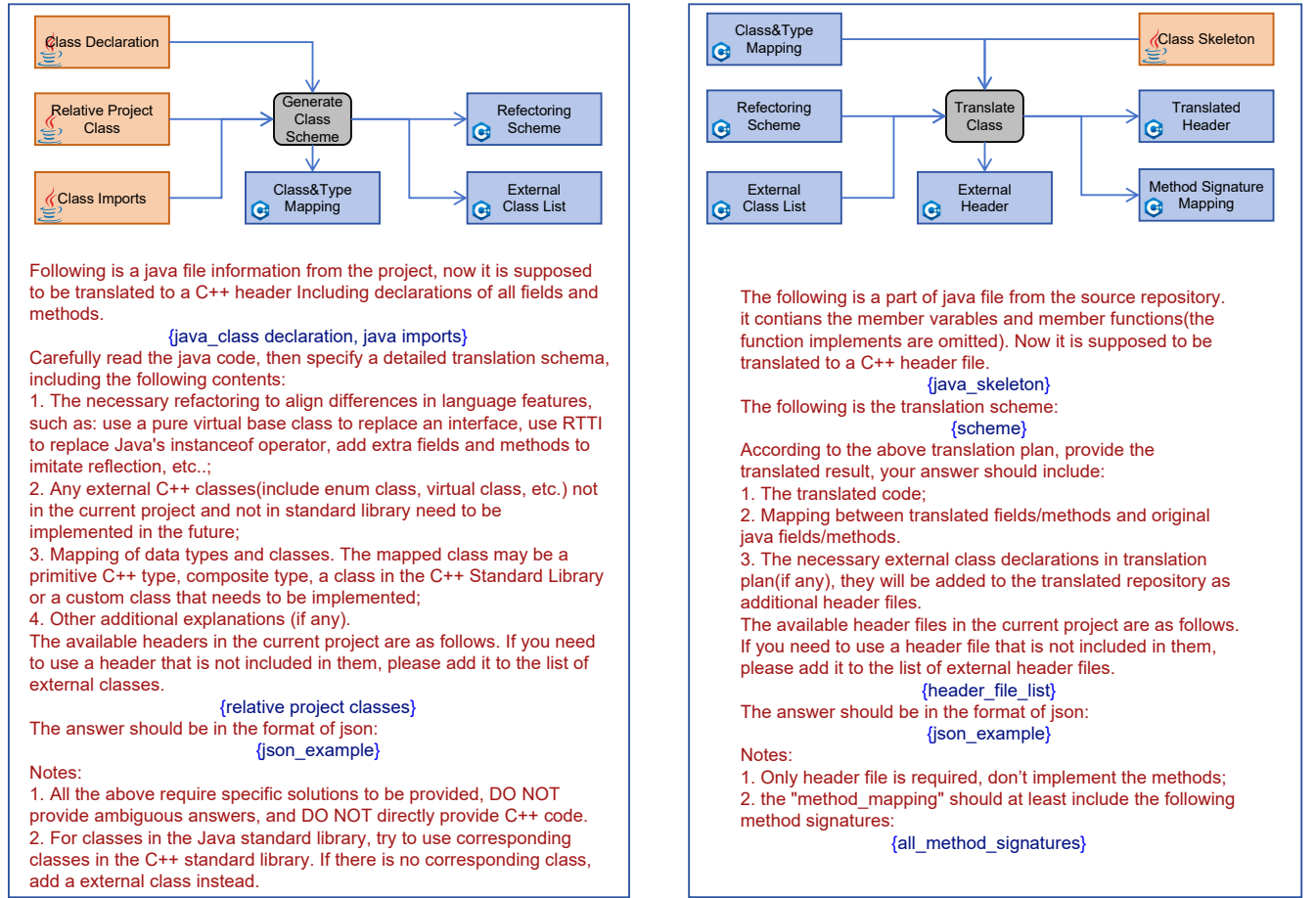


Figure 8: Header-level translation scheme and data flow. The LLM analyzes the Java class structure, generates a translation scheme specifying type mappings and structural changes, then produces the corresponding C++ header file.

- **Implementation strategy:** How to achieve the same behavior in C++ (e.g., replacing Java streams with C++ algorithms, converting iterator-based loops with range-based for loops)
- **Context requirements:** Which already-translated methods to reference as examples
- **Exception handling:** How to convert Java checked exceptions to C++ error handling (e.g., return values, error codes, or C++ exceptions)

Because all callees are translated before their callers, the scheme can explicitly reference specific translation patterns from already-translated methods, promoting consistency across the codebase.

Method Translation. Methods are translated layer by layer. For each method, the LLM receives:

- The original Java method code
- The method's translation scheme
- **Translated code of all callees:** This provides concrete examples of how similar constructs were translated, enabling *pattern-based consistency*

- The translated Header for context on available types and APIs

This context-rich prompting significantly reduces hallucination by providing the LLM with concrete examples of how similar language constructs were translated elsewhere in the project.

4.4 Validation and Repair

The final phase addresses compilation errors through an autonomous agent that iteratively analyzes, diagnoses, and fixes translation defects. This phase bridges the gap between syntactically correct translation and semantically correct, compilable code.

4.4.1 Compilation Agent Architecture. The Compilation Agent operates as a closed-loop system with four core capabilities accessed through a tool-use interface:

- (1) **compile:** Invoke the C++ compiler on a specific source file and capture the output
- (2) **read_file:** Read the current contents of any source or header file
- (3) **write_file:** Replace the contents of a file with modified code

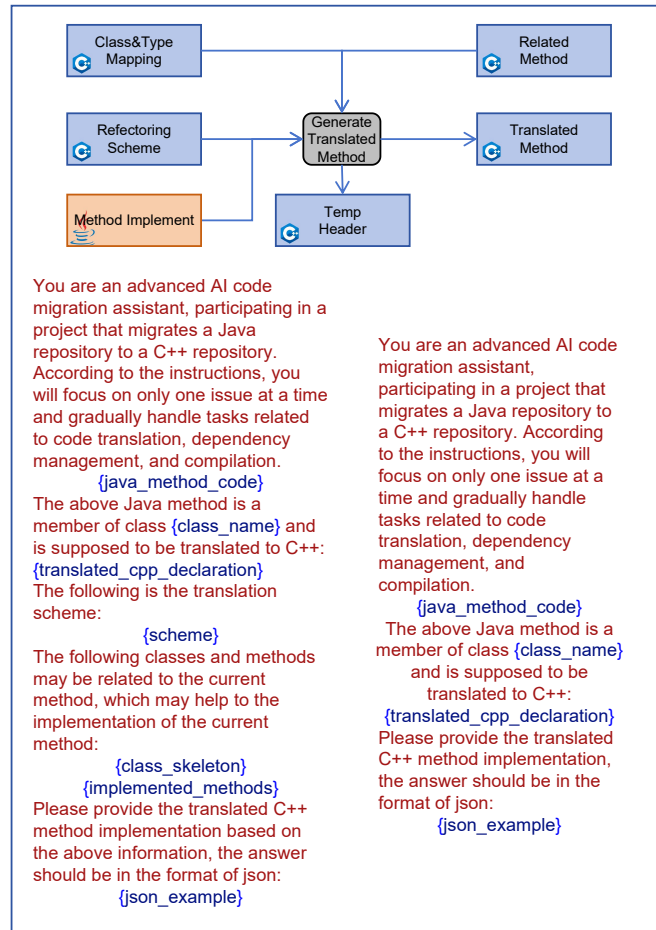
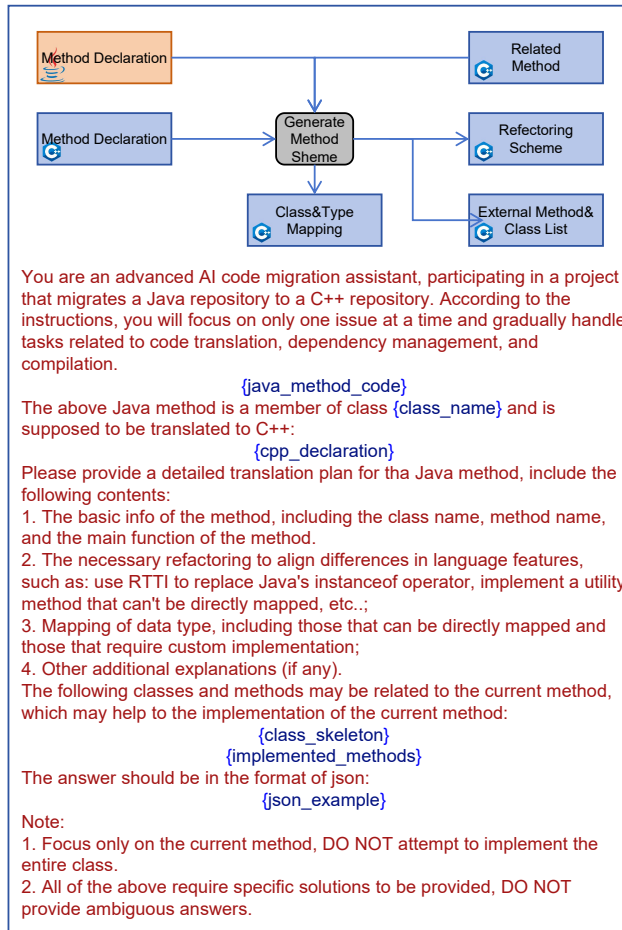


Figure 9: Method-level translation scheme and data flow. The bottom-up translation order ensures that callee methods are translated before callers, enabling context propagation where already-translated code serves as reference.

(4) **create_file**: Generate new header or source files as needed

The agent maintains a conversation history with the LLM, providing compilation errors as feedback and requesting fixes. This interactive process continues until compilation succeeds or maximum attempts are reached.

4.4.2 Iterative Repair Process. For each C++ source file in dependency order, the agent executes the following loop:

- (1) Invoke compile on the target source file
- (2) If compilation succeeds, proceed to the next file
- (3) Otherwise, provide the compilation error output to the LLM
- (4) The LLM analyzes the error and generates a response containing:
 - A diagnosis of the root cause
 - A sequence of tool calls to fix the issue (e.g., `read_file` to examine code, `write_file` to apply corrections, `create_file` to add missing headers)
 - A status indicator for whether further iterations are needed
- (5) Execute the requested tool calls

(6) Repeat from step 1

This approach enables automatic resolution of common translation errors including:

- Missing header includes
- Type mismatches and implicit conversion issues
- Template instantiation errors
- Linker errors due to missing definitions

4.5 Key Design Features

4.5.1 Bottom-Up Translation with Context Propagation. The bottom-up translation order is the cornerstone of our approach, enabling context propagation that significantly improves translation quality. By translating callees before callers, we ensure that:

- (1) **Consistent API usage**: When translating a caller method, the LLM sees the exact signatures and usage patterns of already-translated callees, eliminating mismatches in function calls, parameter passing, and return value handling.
- (2) **Pattern reinforcement**: Common translation patterns (e.g., how `Optional<T>` is converted, how exceptions are

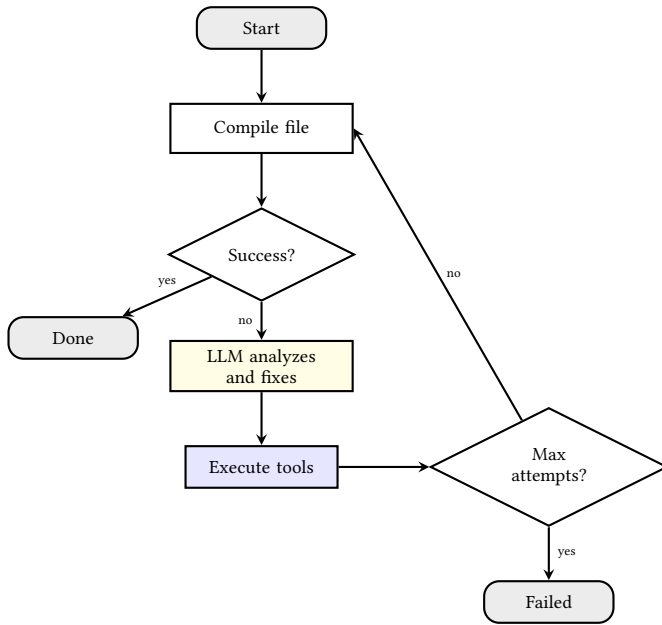


Figure 10: The compilation agent’s iterative repair process.

handled) are established early in leaf methods and consistently propagated through the call graph, reducing ad-hoc decisions.

- (3) **Incremental complexity:** Simple methods (often with few dependencies) are translated first, establishing project-wide conventions before tackling complex methods with deep call stacks.

This ordering is derived automatically from the call graph using topological sort (Kahn’s algorithm), which groups methods into dependency layers where all methods in layer i depend only on methods in layers 1 through $i - 1$.

4.5.2 Dual-Pass Translation with Scheme Generation. Each translation unit (Header and Method) undergoes two passes:

- (1) **Scheme Pass:** High-level planning without generating code. The LLM produces a structured document analyzing the translation requirements and specifying the strategy.
- (2) **Code Pass:** Implementation guided by the scheme. The LLM generates actual C++ code following the strategy established in the scheme.

The two-pass approach provides several critical benefits:

- **Separation of concerns:** Strategic decisions (type mappings, refactoring needs, error handling approach) are made separately from tactical implementation, reducing cognitive load on the LLM.
- **Explicit reasoning:** The scheme serves as an explanation of the translation approach, making the process more interpretable and enabling verification of design decisions before code generation.

- **Error localization:** When translation errors occur, the scheme provides context for understanding the LLM’s intent, facilitating more effective error diagnosis and correction.
- **Template for consistency:** The scheme establishes conventions that can be referenced when translating related code (e.g., all methods in a class use the same type mappings specified in the Header scheme).

4.5.3 Hierarchical Graph Representation. The bipartite graph model (Headers and Methods) provides a clean separation between structural and behavioral aspects of the codebase:

- **Structural isolation:** Header translation focuses exclusively on type definitions, field declarations, and API contracts, without the complexity of method implementations.
- **Behavioral independence:** Method translation operates on implementation logic separately from type definitions, with the translated Header providing a stable API contract.
- **Explicit dependencies:** The graph captures both structural relationships (inheritance, type usage through imports) and behavioral relationships (method calls), enabling systematic translation ordering.

This representation maps naturally to the C++ compilation model (header files and implementation files), while accommodating Java’s compilation unit structure (where multiple classes may reside in one source file).

4.5.4 Heterogeneous Translation with Structural Adaptation. Unlike traditional code translation approaches that aim for syntactic one-to-one mapping, the translated codebase may exhibit structural differences from the source while preserving functional equivalence. This design philosophy is enabled by two key features:

- **Scheme-driven refactoring:** During scheme generation, the LLM can propose structural transformations that better align with target language idioms. These refactoring decisions are made before code generation, allowing systematic reorganization of the codebase structure.
- **Header-level flexibility:** The Header translation phase operates at the type-system level, where the LLM has broad freedom to redesign APIs, reorganize inheritance hierarchies, or adopt entirely different architectural patterns (e.g., replacing inheritance with templates, converting classes to namespaces).

This approach is particularly advantageous for cross-paradigm translation (e.g., Java to C++), where mechanical preservation of source structure often results in non-idiomatic code. For example:

- Java’s single-inheritance class hierarchy may be restructured using C++ multiple inheritance or CRTP (Curiously Recurring Template Pattern)
- Runtime polymorphism based on interfaces may be partially replaced with compile-time polymorphism using templates
- Reflection-based designs may be restructured to use compile-time type traits and constexpr metaprogramming
- Exception-heavy error handling may be converted to RAII-based resource management with optional return values

By granting the LLM freedom to adapt project structure to target language conventions, heterogeneous translation produces more maintainable and idiomatic code than rigid structure-preserving approaches.

5 Evaluation

This section presents a comprehensive evaluation of TOOL. We first articulate our research questions, then describe the experimental setup, datasets, and evaluation metrics. We then present both quantitative results and qualitative analysis through case studies.

5.1 Research Questions

Our evaluation is guided by the following research questions:

- **RQ1:** To what extent can TOOL successfully translate Java projects to compilable C++ code?
- **RQ2:** What is the individual contribution of each key component in TOOL?
- **RQ3:** What types of translation errors persist after the automated repair process?

5.2 Experimental Setup

5.2.1 Implementation Configuration.

Translation Framework. We implemented TOOL using Python 3.10, with the following key configurations:

- **LLM Backend:** We use Qwen-2.5 72B, DeepSeek-V3, GPT-4o as the LLM backend. All LLM calls use structured outputs with JSON Schema validation to ensure reliable parsing of translation schemes and generated code.
- **Static Analysis:** Java source code is parsed using a custom Java parser tool integrated with java-callgraph2 for call graph extraction. The parser outputs project structure and method call relationships in JSON format.
- **Compilation Agent:** The agent uses clang++ (version 20.1.8) as the C++ compiler with C++17 standard. Maximum repair attempts are set to 10 per source file.

5.2.2 Baselines. To evaluate the effectiveness of TOOL, we compare against the following baselines:

- **Direct Translation (DT):** Files are translated sequentially without call graph analysis. Each .java file is translated independently to a .cpp file using the same LLM, with no cross-file context or scheme generation.
- **Top-Down Translation (TDT):** Headers are translated first (similar to TOOL), but methods are translated in top-down order (caller before callee) without the bottom-up dependency ordering.
- **No-Scheme Translation (NST):** Uses the same call graph and bottom-up ordering as TOOL, but skips the scheme generation pass, directly generating code from source.

All baselines use the same LLM model and compilation agent configuration for fair comparison.

5.3 Dataset

5.3.1 Project Selection. We selected open-source Java projects from GitHub repositories. Projects span multiple domains including concurrent programming, data parsing, testing frameworks and web frameworks. This diversity tests the framework's ability to handle different programming patterns and libraries.

Since a complete real-world project is too large to translate directly, we first extract a functional file and then find all files related to it to obtain different translation modules. After the translation is completed, we use the number of files in each translation module that can be successfully compiled as a measure of the translation's effectiveness. **Language Feature Coverage:** Projects collectively exercise various Java features that present translation challenges:

- Generics and wildcard types
- Inheritance hierarchies (both single inheritance and interface implementation)
- Exception handling (checked exceptions, try-catch-finally blocks)
- Anonymous classes and lambda expressions
- Standard library usage (Collections Framework, I/O streams, concurrency utilities)

5.3.2 Dataset Statistics. Table 5 summarizes key statistics of the dataset.

5.4 Evaluation Metrics

5.4.1 Primary Metric: Compilation Success Rate. The primary metric for evaluating translation effectiveness is the **compilation success rate**, defined as:

$$\text{Success Rate} = \frac{\text{Number of successfully compiled C++ files}}{\text{Total number of generated C++ files}} \times 100\% \quad (1)$$

5.5 Quantitative Results (RQ1)

5.5.1 Overall Translation Performance. Table 6 presents the overall translation performance of TOOL across all projects.

Table 5: Dataset Statistics

Translation Module	Main Class	Source Project	Category	Java Files	Methods
Module1	BulkheadBuilder	failsafe	fault tolerance	70	97
Module2	CircuitBreakerExecutor	failsafe	fault tolerance	64	84
Module3	DelayablePolicy	failsafe	fault tolerance	58	86
Module4	ExecutionImpl	failsafe	fault tolerance	54	105
Module5	FailsafeExecutor	failsafe	fault tolerance	68	131
Module6	FailurePolicy	failsafe	fault tolerance	60	94
Module7	RateLimiterBuilder	failsafe	fault tolerance	68	102
Module8	RateLimiterExecutor	failsafe	fault tolerance	61	93
Module9	RetryPolicyConfig	failsafe	fault tolerance	56	97
Module10	SyncExecutionImpl	failsafe	fault tolerance	59	97
Module11	TimeoutExecutor	failsafe	fault tolerance	55	82
Module12	UndeclaredThrowableStrategy	failsafe	fault tolerance	31	56
Module13	AdviceListenerTestCase	jvm-sandbox	AOP framework	65	152
Module14	ClassStructureByChildClassTestCase	jvm-sandbox	AOP framework	44	76
Module15	DebugLogExceptionModule	jvm-sandbox	AOP framework	31	85
Module16	EventWatchBuilderTestCase	jvm-sandbox	AOP framework	38	122
Module17	OrGroupFilter	jvm-sandbox	AOP framework	27	86
Module18	ProgressPrinter	jvm-sandbox	AOP framework	14	11
Module19	TestIssues217	jvm-sandbox	AOP framework	55	129
Module20	TestIssues256	jvm-sandbox	AOP framework	40	118
Module21	ReadRowHolder	easyexcel	Excel processing	10	13
Module22	ReadSheetHolder	easyexcel	Excel processing	28	49
Module23	Cookie	httpclient	HTTP client	20	79
Module24	JSONParser	hutool	utility library	17	17
Module25	PerMessageDeflateExtensionTest	Java-WebSocket	WebSocket	39	104
Module26	Draft_6455Test	Java-WebSocket	WebSocket	38	87
Total				1293	2440

Table 6: Overall Translation Performance Summary

compile unit	file count	cpp file count	method count	success count	success rate
AdviceListenerTestCase	65	18	205	14	77.78%
BulkheadBuilder	70	15	140	10	66.67%
CircuitBreakerExecutor	64	13	141	12	92.31%
ClassStructureByChildClassTestCase	44	10	89	9	90.00%
Cookie	20	7	93	4	57.14%
DebugLogExceptionModule	31	9	138	8	88.89%
DelayablePolicy	58	13	123	10	76.92%
Draft_6455Test	38	13	109	10	76.92%
EventWatchBuilderTestCase	38	11	174	10	90.91%
ExecutionImpl	54	12	149	10	83.33%
FailsafeExecutor	68	14	170	9	64.29%
FailurePolicy	60	13	132	11	84.62%
Issue231Test	61	15	139	13	86.67%
Issue260Test	62	14	123	12	85.71%
JSONParser	17	4	20	4	100.00%
OrGroupFilter	27	8	138	7	87.50%
PerMessageDeflateExtensionTest	39	14	123	9	64.29%
ProgressPrinter	14	2	37	2	100.00%
RateLimiterBuilder	68	15	155	12	80.00%
RateLimiterExecutor	61	14	146	7	50.00%
ReadRowHolder	10	2	16	2	100.00%
ReadSheetHolder	28	5	51	4	80.00%
RetryPolicyConfig	56	12	133	9	75.00%
SyncExecutionImpl	59	12	142	9	75.00%
TestIssues217	55	13	193	10	76.92%
TestIssues256	40	12	170	10	83.33%
TimeoutExecutor	55	12	116	10	83.33%
UndeclaredThrowableStrategy	31	6	62	4	66.67%
TOTAL ALL PROJECTS	1293	308	3427	241	78.24%

References

- [1] Xinyun Chen, Chang Liu, and Dawn Song. 2018. Tree-to-tree neural networks for program translation. *Advances in neural information processing systems* 31 (2018).
- [2] Yuyan Chen, Qiang Fu, Yichen Yuan, Zhihao Wen, Ge Fan, Dayiheng Liu, Dongmei Zhang, Zhixu Li, and Yanghua Xiao. 2023. Hallucination detection: Robustly discerning reliable answers in large language models. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 245–255.
- [3] Mehdi Drissi, Olivia Watkins, Aditya Khant, Vivaswat Ojha, Pedro Sandoval, Rakia Segev, Eric Weiner, and Robert Keller. 2018. Program language translation using a grammar-driven tree-to-tree model. *arXiv preprint arXiv:1807.01784* (2018).
- [4] Jaemin Hong and Sukyoung Ryu. 2025. Type-migrating C-to-Rust translation using a large language model. *Empirical Software Engineering* 30, 1 (2025), 3.
- [5] Ali Reza Ibrahimzade, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2454–2476.
- [6] javaparser. 2025. Java Parser Tools for Your Java Code Analyse. <https://javaparser.org/>. Accessed: 2026-02-22.
- [7] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.
- [8] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the evaluation of neural code translation: Taxonomy and benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1529–1541.
- [9] Fang Liu, Yang Liu, Lin Shi, Zhen Yang, Li Zhang, Xiaoli Lian, Zhongqi Li, and Yuchi Ma. 2026. Beyond Functional Correctness: Exploring Hallucinations in LLM-Generated Code. *arXiv preprint arXiv:2404.00971v3* (2026). arXiv:2404.00971 [cs.SE]
- [10] Jiaqi Liu, Fengming Zhang, Xin Zhang, Zhiwen Yu, Liang Wang, Yao Zhang, and Bin Guo. 2024. hmcodeTrans: Human-machine interactive code translation. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1163–1181.
- [11] Feng Luo, Kexing Ji, Cuiyun Gao, Shuzheng Gao, Jia Feng, Kui Liu, Xin Xia, and Michael R Lyu. 2025. Integrating Rules and Semantics for LLM-Based C-to-Rust Translation. *arXiv preprint arXiv:2508.06926* (2025).
- [12] Aniketh Malyala, Katelyn Zhou, Baishakhi Ray, and Saikat Chakraborty. 2023. On ML-based program translation: perils and promises. In *2023 IEEE/ACM 45th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. IEEE, 60–65.
- [13] Potsawee Manakul, Adian Liusie, and Mark Gales. 2023. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*. 9004–9017.
- [14] Vikram Nitin, Rahul Krishna, and Baishakhi Ray. 2024. Spectra: Enhancing the code translation ability of language models by generating multi-modal specifications. *arXiv preprint arXiv:2405.18574* (2024).
- [15] Y. Ou, Z. Zhang, et al. 2025. K3Trans: Evolving Triple Knowledge-Augmented LLMs for Code Translation in Repository Context. *arXiv preprint arXiv:2503.18305* (2025). <https://arxiv.org/html/2503.18305v3>
- [16] Rangeet Pan, Ali Reza Ibrahimzade, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [17] Vipula Rawte, Amit Sheth, and Amitava Das. 2023. A survey of hallucination in large foundation models. *arXiv preprint arXiv:2309.05922* (2023).
- [18] Sohini Roychowdhury. 2024. Journey of hallucination-minimized generative ai solutions for financial decision makers. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 1180–1181.
- [19] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanasot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. *Advances in neural information processing systems* 33 (2020), 20601–20611.
- [20] surveyssystem. 2025. Clang: a C language family frontend for LLVM. <https://clang.llvm.org/>. Accessed: 2026-02-22.
- [21] surveyssystem. 2025. Sample Size Calculator. <https://www.surveysystem.com/sscalc.htm>. Accessed: 2026-02-22.
- [22] Yanli Wang, Yanlin Wang, Suiquan Wang, Daya Guo, Jiachi Chen, John Grundy, Xilin Liu, Yuchi Ma, Mingzhi Mao, Hongyu Zhang, et al. 2024. Repotransbench: A real-world benchmark for repository-level code translation. *arXiv preprint arXiv:2412.17744* (2024).
- [23] Jia-Yu Yao, Kun-Peng Ning, Zhen-Hui Liu, Mu-Nan Ning, Yu-Yang Liu, and Li Yuan. 2023. Llm lies: Hallucinations are not bugs, but features as adversarial examples. *arXiv preprint arXiv:2310.01469* (2023).
- [24] Zhiqiang Yuan, Wenjun Mao, Zhuo Chen, Xiyue Shang, Chong Wang, Yiling Lou, and Xin Peng. 2025. Project-Level C-to-Rust Translation via Synergistic Integration of Knowledge Graphs and Large Language Models. *arXiv preprint arXiv:2510.10956* (2025).
- [25] Xing Zhang, Jiaheng Wen, Fangkai Yang, Pu Zhao, Yu Kang, Junhao Wang, Maoquan Wang, Yufan Huang, Elsie Nallipogu, Qingwei Lin, et al. 2025. Skeleton-Guided-Translation: A Benchmarking Framework for Code Repository Translation with Fine-Grained Quality Evaluation. *arXiv preprint arXiv:2501.16050* (2025).
- [26] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 481–503.